

CX 4220/CSE 6220 High Performance Computing (Spring 2025)
Programming Assignment 2
Due: March 07, 2025

1 Problem Statement

In this assignment you will develop your own MPI communication primitives in C/C++. You are expected to write your code from scratch and you can only make use of point-to-point MPI instructions `MPI_Send` and `MPI_Recv` (no `MPI_Isend`/`MPI_Irecv`). You may assume that the length of any array won't be larger than `INT32_MAX`.

2 MPI Communication Primitives

1. **AllReduce** - AllReduce applies a reduction calculation and make the reduction result available to all MPI processes involved. The reduction operation is done element-wise.
 - Assume the processors can only communicate via a **hypercube network**.
 - Should work on any number of processors (correctness test on 15 and 24 processors).
 - The reduction operation is `MPI_SUM`
2. **ManytoMany** - Each processor sends a chunk of its data of variable sizes to every other processor. The received data in every processor is appended based on the rank of the sending processor (0 to p-1).
 - Must use the **arbitrary** communication pattern
 - Should work on any number of processors (correctness test on 15 and 24 processors).

3 Code Framework

3.1 Input Format

All input are space-separated text files. Example input files are included in the folder `small_examples`.

- **AllReduce Input:**

```
<number of processors to be used = p>
<number of elements in each processor = n>
<length-n input array of PE 0>
<length-n input array of PE 1>
...
<length-n input array of PE p-1>
```

- **ManyToMany Input:**

```
<number of processors to be used = p>
<p integers specifying # of elements PE 0 is sending to each PE>
<p integers specifying # of elements PE 1 is sending to each PE>
...
<p integers specifying # of elements PE P-1 is sending to each PE>
<input array of PE 0>
<input array of PE 1>
...
<input array of PE p-1>
```

3.2 Download Files

Download the PA2 files from this link (or)

Use `git clone https://github.com/gtcse6220-s25/pa2.git`

1. **Makefile**
2. **custom_collectives.cpp** - Develop your code in this file. This is your **submission file**.
Note:- You need to only need to submit the **custom_collectives.cpp** file.
3. **allreduce_main.cpp & manytomany_main.cpp** - The driver file for your code. You can make changes in the driver file for your own testing and custom test cases, but make sure that the submitted 'custom_collectives.cpp' file runs with both of the original main file.
4. **custom_collectives.h** - This file contains the function definitions and must not be modified.
5. **small_examples** - This folder contains small example input files.
6. **autograder_gendata.sh** - This file calls Python scripts to generate large input data for simple test cases in **autograder.sh**.
 - **must run this once before using autograder.sh**
 - The generated files will be available in the folder **autograderdata**.
 - Data generation takes a significant amount of time. Avoid modifying the generated data.
 - This script **DOES NOT OVERWRITE EXISTING GENERATED DATA**.
To remove the files, call `make cleanautograder`.
7. **autograder.sh** - This file can be used to check whether you are within the expected performance bounds. Before submitting, make sure your code passes all the tests to obtain the credits.
NOTE: run `chmod +x *.sh` for access denied error

3.2.1 Instructions

1. After downloading all the files, go through each of the files and try to understand the structure.
2. Write your implementation of all the custom functions in **custom_collectives.cpp** file.
3. Use `make allreduce` and `make many2many` to compile the program. An executable **ard** and **m2m** will be created if there are no compilation errors.
4. Request the resources (interactive node) as discussed in the review session using `salloc` and use `module load openmpi` to load the MPI library (which will solve the include/command not found errors, if any).
5. Then use `srun -n <num_processors> ./<executable_name> <input_file>` and to run either of the **ard** and **m2m** executable. **Make sure the number of processors matches the first line in the input file.**

3.3 Deliverables

Note: Please make one submission per team on Gradescope for the code as well as the report.

1. **custom_collectives.cpp** - Make sure this file runs with the original 'main.cpp' file.
2. **Report** - Make sure to list names of all your teammates at the very beginning of your report.
 - Please briefly explain the working of all your custom implementations.
 - Do the theoretical runtime analysis for all your custom implementations which includes both computation and communication cost.
 - Create 2 runtime plots comparing MPI runtime against your custom implementation on varying number of processors $p = 4, 8, 16, 24$ for fixed size problem of $n \approx 10^7$. One plot for each primitives.

3.4 Grading [90 pts]

We will be running your submissions on a Intel(R) Xeon(R) Gold 6226 CPU @ 2.70GHz (24 cores) node with OpenMPI Version 4.1.5.

The correctness tests for grading purposes will be run on problems of size $n \leq 10^7$ with randomly generated send count arrays and data. (The provided autograder only tests one simple case of size $n \approx 10^7$.)

- Team Contract - **5**
- Code - **65**
 - Custom_Allreduce - **20**
 - * 5 points - Correctness on 15 procs
 - * 5 points - Correctness on 24 procs
 - * 5 points - Passing 40s runtime test on 24 procs
 - * 5 points - Passing 30s runtime test on 24 procs
 - Custom_ManyToMany - **45**
 - * 10 points - Correctness on 15 procs
 - * 10 points - Correctness on 24 procs
 - * 15 points - Passing 50s runtime test on 24 procs
 - * 10 points - Passing 40s runtime test on 24 procs
- Report - **20**
 - Explanation of custom implementations - **5**
 - Theoretical runtime analysis of all custom implementations - **5**
 - AllReduce & ManyToMany Plot - **10**

4 Resources

- What is a Makefile and how does it work?: <https://opensource.com/article/18/8/what-how-makefile>
- PACE ICE cluster guide: https://gatech.service-now.com/home?id=kb_article_view&sysparm_article=KB0042102.